

AI Newsroom — System Architecture v4.1

Wine & Food · Italy · USA · China · Full Production-Grade Stack
OpenTelemetry · Vault Agent · Schema Registry · HNSW Vectors · Circuit Breaker · Canary Rollout · Disaster Recovery · Audit Log

OBSERVABILITY & SECRETS MANAGEMENT

■ OpenTelemetry + Grafana Tempo	■ HashiCorp Vault + Agent Sidecar ■			
- Distributed tracing: all Golang + Python services - One trace ID per article — trigger to publish, end to end - W3C TraceContext in RedPanda headers + gRPC metadata - SLO tracking: p95 latency per service in Grafana - Grafana Tempo as trace backend — native Grafana integration	- Vault Agent sidecar: live rotation, no service restart (v4.1) - Credentials written to /vault/secrets/ on tmpfs volume - Least-privilege: each service reads only its own secret path - PostgreSQL + Redis passwords auto-rotate every 30 days - JWT RS256 key rotation uses 15-min overlap window (zero-downtime)			
OTEL SDK GO	OTEL SDK PYTHON	GRAFANA TEMPO	HA RAFT CLUSTER	Vault Agent Sidecar
			AUTO-ROTATION	AUDIT LOG

■ Prometheus + Grafana — Metrics & SLOs	Service	SLI
- Infrastructure: RedPanda lag · Redis memory · PG query p99 · DLQ depth (alert > 0) - AI Quality: rejection rate (< 10%) · correction rate (< 20%) · quality score trend - Circuit breaker state per market · Correction TTL remaining (alert < 1h)	Article generation	p95 end-to-end latency
	Moderation	p95 latency
	Correction fast path	p95 propagation
	API Gateway	Availability
	Embedding batch	p95 per 100 items
PROMETHEUS	GRAFANA	

FRONTEND · AUTH · GATEWAY

HTTPS · JWT Bearer Tokens · All secrets from Vault Agent Sidecar

■ Next.js · Vercel · TypeScript	■ Auth Service · JWT RS256 + Casbin	■■ Caddy · API Gateway
- Server Functions → Caddy gateway (no direct browser API calls) - Multi-language rendering with next-intl (IT/EN/ZH) - Role-aware UI: Admin / Editor / Journalist / Viewer - Editorial corrections → dual fast+slow path	- JWT RS256 — private key from Vault Agent, 15min access / 7d refresh - Casbin RBAC — roles scoped per market (IT/EN/ZH) - Shared JWT middleware across all Golang services - Audit log write on every role change + token revocation	/api/auth/* → Auth (8081) · /api/articles/* → Agent (8082) /api/analytics/* → Analytics (8083) · /api/editor/* → Learner (8084) - JWT validation on every route except /login and /refresh - gRPC ↔ REST transcoding · Auto HTTPS · Load balancing
VERCEL	SSR	I18N
JWT HTTPONLY COOKIES		
GOLANG	CASBIN + POSTGRESQL	REDIS B
DOCKER		
GRPC + REST		
JWT GUARD		

gRPC · OTel trace context propagated in all metadata · /health + /ready on :8090 per service

CORE SERVICES

■ Learner	■ Agent Enhanced	■ Analytics	■■ Moderation	■ Correction Processor ■
- Scrapes journalist agents (wine/food) - Updates knowledge graph (slow path) - Embedding batching: 100/call → HNSW (v4.1) - Ingests corrections + rejections - Flags stale vectors after corrections - DEL Redis correction key after applying - Publishes graph.updated	- fetch_memory → Redis short-term - fetch_corrections → fast path (48h TTL) - Semantic search HNSW (skip stale) - Circuit breaker + fallback (v4.1) - Token bucket rate limit per market - Max 2 concurrent LangGraph runs/market - Market-specific system prompts	- Track article performance per market - Sliding window trend detection - Agent quality scoring over time - Publishes topic.trending events - Exposes SLO metrics to Prometheus	- Cultural sensitivity per market (IT/EN/ZH) - Factual accuracy vs knowledge graph - LLM quality scoring - Rejected → triggers fast + slow path - Audit log write on every decision - DLQ on failure — never loses articles	- Consumes editor.correction events - Writes override to Redis immediately - TTL: 48h (raised from 24h) (v4.1) - Flags factual vectors as stale - Stores provenance + correction type - Audit log write on every correction
GOLANG	OTEL	GOLANG	OTEL	
		PYTHON	LANGCHAIN	OTEL

■ Dual feedback loops: Fast path (Redis, seconds) + Slow path (knowledge graph, hours) for both editor corrections and moderation rejections. Agent benefits immediately on next generation cycle.

EVENT BROKER · RedPanda + Schema Registry + Dead Letter Queues ■ NEW v4.1

Event	Flow	Event	Flow
graph.updated	Learner → Agent	topic.trending	Analytics → Agent
article.generated	Agent → Moderation	article.approved	Moderation → Sanity (idempotency: article.id)
article.published	Sanity → Analytics	editor.correction	Frontend → Learner + Correction
moderation.rejected	Moderation → Learner + Correction	user.data.deletion.requested	Auth → All services (GDPR v4.1)

■ Dead Letter Queues				■ Schema Registry ■	
*.dlq topic for every consumer topic - Retry: 5s → 30s → 2m → DLQ + alert - Grafana alert: DLQ depth > 0 - CLI: cmd/dlq-tool list/inspect/replay/discard - Runbook: ops/DLQ-handling.md				- Confluent-compatible Schema Registry - All events validated at produce + consume - Schemas stored in infra/schemas/ - Breaking changes require new event version - CI/CD: schema compatibility check on every PR	
3 BROKER REPLICAS		RF=3	OTEL HEADERS	AVRO / JSON SCHEMA	CONFLUENT-COMPAT

DATA LAYER

■ PostgreSQL			■ Redis 8+ · Multi-Model 5 roles			■ Sanity.io · Content Lake			
• Knowledge graph: journalists, articles, topics, edges • Casbin RBAC policies • Users + immutable audit_log (append-only, v4.1) • Editorial calendar + workflow state • Migrations: golang-migrate in infra/migrations/ (v4.1)			• Vector Store: HNSW index per market · KNN · stale flags (v4.1) • Short-term Memory: topics/sources/angles · 3-14d TTL • Fast Corrections: editor overrides · 48h TTL (v4.1) • Rate Limiter: token bucket per market · 10 tokens/30s refill • Cache + Queue: response cache · async jobs · JWT blacklist			• Receives only moderation-approved articles • Idempotency key: article.id on all API calls (v4.1) • Editorial calendar · Human editor review queue • Correction webhooks → RedPanda → dual path • Multi-language variants (IT/EN/ZH) per article			
2 REPLICAS		PATRONI HA	SENTINEL HA		AOF + RDB → S3	HNSW INDEX	GROQ API	CDN	MULTI-LANGUAGE

CI/CD · INFRASTRUCTURE · DISASTER RECOVERY

■ GitHub Actions CI/CD + Argo Rollouts ■			■■ Infrastructure (Microservices from Day 1)				■ Disaster Recovery (NEW v4.1) ■		
• Per-service workflow with path filtering (8 workflows) • PR → Test only · Merge → migrate → staging (auto) • Production: canary 5% → 50% → 100% (Argo Rollouts v4.1) • Auto-rollback: error > 1% OR p99 latency +50% (v4.1) • Go: golangci-lint + race detector + buf lint • Python: ruff + mypy + pytest 80% + schema tests			• Docker all services (Go + Python) + /health endpoints • Kubernetes via Helm — minimal replicas initially • K8s from day one (monolith-first removed — contradictory) • Multi-replica HA all critical services • CLI: cmd/dlq-tool — list, inspect, replay, discard				• PostgreSQL: WAL archiving → S3 continuous + daily pg_dump (15min RPO) • Redis: daily RDB snapshot → S3 + AOF persistence (24h RPO) • Vault: daily Raft snapshot → S3 (24h RPO) • All backups encrypted at rest (AWS KMS) • Restore runbook: ops/DR.md · Quarterly drills mandatory		
DOCKER		TERRAFORM		HELM	K8S	S3 BACKUPS		15MIN RPO PG	QUARTERLY DR
8 WORKFLOWS		CANARY ROLLOUT		STAGING GATE					

• OpenTelemetry + Tempo		• HashiCorp Vault + Agent		• Next.js Frontend		• Auth · JWT + Casbin		• Core Services (Golang)		• Agent (Python)		• Correction Processor	
• RedPanda + Schema Registry + DLQ		• PostgreSQL + golang-migrate		• Redis 8+ HNSW		• Sanity Content Lake		• GitHub Actions + Argo Rollouts		• S3 Disaster Recovery		• Audit Log (v4.1)	